

Cosmic Choreography: Technical Documentation & Mathematical Foundation

A Comprehensive Engineering Guide to Planetary Path Tracing

Author: ChandraKamal

System Architecture: Single-File Web Application (HTML5 / Canvas / Vanilla JS)

Deployment Target: GitHub Pages Static Infrastructure

1. Executive System Overview

Cosmic Choreography is a deterministic, 100% client-side rendering engine that visualizes complex orbital resonance and parametric spirograph patterns created by the relative movements of celestial bodies.

The core philosophy of the engine is mathematical fidelity coupled with visual aesthetics. Unlike generic spirograph visualizers that use linear gear ratios, this engine implements true Keplerian orbital geometry—incorporating orbital eccentricity and spatial inclination—to map out the harmonic relationships of the solar system.

Architectural Blueprint

The system utilizes a dual-canvas visual stack running inside a hardware-accelerated rendering loop:

1. **#starsCanvas (Background Layer):** A static background layer that procedurally renders an organic deep space environment using radial gradient light maps and alpha-blended stellar distributions.
2. **#simCanvas (Workspace Layer):** A dynamic layer rendering at up to 60 frames per second using a high-fidelity recursive matrix array to store tracking coordinates. It uses a **screen** mix-blend mode to maximize line luminescence against the deep space backdrop.

2. The Mathematical Foundation

To achieve scientific depth, the simulator resolves several astrophysical formulas sequentially on every frame animation tick using standard coordinate transformations.

Step 1: Angular Temporal Progression

Time (t) increments smoothly based on the step value of the simulation speed slider. The angular position of a planet (its orbital angle) is derived directly from its average angular velocity and its total orbital period (T) in Earth years:

$$\text{Angle} = (2 * \pi / T) * \text{time}$$

(Where 'pi' is approximately 3.14159, and T is the planet's unique orbital length).

Step 2: Resolving Keplerian Eccentricity

Real planetary orbits are not perfect circles; they are ellipses with the host star sitting at one of the focal points. To calculate the instantaneous distance (r) of a planet from the Sun at any given angle, the engine processes the polar coordinate equation of an ellipse:

$$r = [a * (1 - e^2)] / [1 + e * \cos(\text{Angle})]$$

Where:

- **a** = Semi-major axis of the orbit (the maximum radius in Astronomical Units).
- **e** = Orbital eccentricity parameter (a value between 0 and 1 detailing how squished the oval is).
- **cos** = The standard mathematical cosine function.

Step 3: 3D Inclination Projection (Z-Depth)

Planets do not occupy a perfectly flat plane; each orbit is tilted relative to the solar system's ecliptic plane by an inclination angle (i). To simulate spatial depth, the coordinate matrix calculates an artificial three-dimensional displacement (z):

- **x** = **r** * **cos(Angle)**

- $y = r * \sin(\text{Angle})$
- $z = r * \sin(i) * \sin(\text{Angle})$

The resulting variable **z** serves as a spatial depth metric. It is mapped directly to the canvas rendering context's transparency channel, making lines fade out subtly as a planet travels far above or below the flat plane of reference.

Step 4: Reference Frame Vector Transformations

The simulator transforms position coordinates depending on whether the observer's frame of reference is Heliocentric or Geocentric (Planet 1 Centered).

- **Heliocentric State:** The Sun position is locked at local center (0, 0). The positions of Planet 1 and Planet 2 are drawn directly from their raw coordinates:
 - *Sun Position:* [0, 0]
 - *Planet 1 Position:* [x1, y1]
 - *Planet 2 Position:* [x2, y2]
- **Planet 1 Centered State:** Planet 1 is structurally pinned at the local center coordinate space (0, 0). To maintain relative physics, the positions undergo an inversion translation shift:
 - *Planet 1 Position:* [0, 0]
 - *Sun Position:* [-x1, -y1]
 - *Planet 2 Position:* [x2 - x1, y2 - y1]

Step 5: Synodic Period & Relative Proximity

The system monitors structural alignments by evaluating the **Synodic Period**—the absolute time required for two bodies to return to the exact same relative angular orientation facing the Sun:

$$\text{Synodic Period} = (T1 * T2) / \text{Absolute_Value}(T2 - T1)$$

The raw spatial distance between the two tracking points is evaluated continuously using standard Euclidean metrics, which drive the real-time line graph at the bottom of your screen:

$$\text{Distance} = \text{Square_Root}((x_2 - x_1)^2 + (y_2 - y_1)^2)$$

3. Code Block Explanations

Block 1: The Positional Computer Engine

This function maps raw timeline values to calculated vector coordinates, taking eccentricity and inclination properties directly into account.

```
function getPlanetPos(pData, t) {  
  // 1. Convert temporal velocity down to radians  
  const theta = (2 * Math.PI / pData.T) * t;  
  
  // 2. Execute polar ellipse equation to find current  
focal radius  
  const r = (pData.R * (1 - pData.e * pData.e)) / (1 +  
pData.e * Math.cos(theta));  
  
  // 3. Project structural orbital plane coordinates  
  const x = r * Math.cos(theta);  
  const y = r * Math.sin(theta);  
  
  // 4. Extract out-of-plane Z displacement using  
inclination angle  
  const z = r * Math.sin(pData.i) * Math.sin(theta);  
  
  return { x, y, z, theta, r };  
}
```

Block 2: The Main Animation Loop & Vector Mapping

The core execution thread steps time forward, resolves reference transformations, updates data arrays, and renders path trajectories to the canvas space.

```
function draw() {
    const p1Data = planets[p1Select.value], p2Data =
planets[p2Select.value];
    const frame = refFrameSelect.value;

    if (isPlaying) {
        time += parseFloat(speedSlider.value);

        // 1. Grab raw elliptical coordinate calculations
        let rawP1 = getPlanetPos(p1Data, time);
        let rawP2 = getPlanetPos(p2Data, time);

        let sunPos, p1Pos, p2Pos;

        // 2. Transform spatial coordinates based on chosen
reference frame
        if (frame === 'heliocentric') {
            sunPos = { x: 0, y: 0, z: 0 };
            p1Pos = rawP1;
            p2Pos = rawP2;
        } else {
            p1Pos = { x: 0, y: 0, z: 0 };
            sunPos = { x: -rawP1.x, y: -rawP1.y, z: -rawP1.z
};
            p2Pos = { x: sunPos.x + rawP2.x, y: sunPos.y +
rawP2.y, z: sunPos.z + rawP2.z };
        }
    }
```

```
// 3. Evaluate Euclidean distance profile matrix
const dist = Math.hypot(p1Pos.x - p2Pos.x, p1Pos.y -
p2Pos.y);

// 4. Map spatial proximity boundaries to HSL
spectrum spaces
const maxDistPossible = p1Data.R * (1+p1Data.e) +
p2Data.R * (1+p2Data.e);
const minDistPossible = Math.abs(p1Data.R *
(1-p1Data.e) - p2Data.R * (1-p2Data.e));
const normDist = Math.max(0, Math.min(1, (dist -
minDistPossible) / (maxDistPossible - minDistPossible ||
1)));
const hue = 180 + (normDist * 100); // Transitions
from Cyan to Purple

// 5. Track Z-depth average to calculate lines
opacity
const zFade = (p1Pos.z + p2Pos.z) / 2;

// 6. Push verified vector packet directly to history
storage arrays
drawnLines.push({ x1: p1Pos.x, y1: p1Pos.y, x2:
p2Pos.x, y2: p2Pos.y, hue: hue, z: zFade });
distanceHistory.push(dist);

// Manage strict length constraints for visualization
boundaries
if (distanceHistory.length > 200)
distanceHistory.shift();
```

```

        if (autoRemoveCheckbox.checked && drawnLines.length >
parseInt(maxLinesSlider.value)) {
            drawnLines.shift();
        }
    }

    // 7. Render loops loop context elements sequentially
    onto workspace layer
    renderScenePack(frame, p1Data, p2Data);

    animationId = requestAnimationFrame(draw);
}

```

Block 3: Procedural Nebula Generator

To create a high-fidelity deep space visual field, this system uses multiple overlapping canvas operations to build soft nebulae gas layers instead of basic randomized noise loops.

```

function generateCinematicSpace() {
    starsCtx.clearRect(0, 0, starsCanvas.width,
starsCanvas.height);

    // Layer 1: Multi-pass radial gradient maps for
    volumetric gas simulation
    starsCtx.globalCompositeOperation = 'screen';
    for(let i = 0; i < 4; i++) {
        const cx = Math.random() * starsCanvas.width;
        const cy = Math.random() * starsCanvas.height;
        const r = Math.random() * 400 + 300;

        const grad = starsCtx.createRadialGradient(cx, cy, 0,

```

```

cx, cy, r);
    const hue = Math.random() > 0.5 ? (Math.random() * 40
+ 220) : (Math.random() * 30 + 270);

    grad.addColorStop(0, `hsla(${hue}, 60%, 40%, 0.08)`);
    grad.addColorStop(0.5, `hsla(${hue}, 50%, 30%,
0.03)`);
    grad.addColorStop(1, `hsla(${hue}, 50%, 20%, 0)`);

    starsCtx.fillStyle = grad;
    starsCtx.beginPath(); starsCtx.arc(cx, cy, r, 0,
Math.PI * 2); starsCtx.fill();
}

// Layer 2: Alpha-modulated stellar distribution array
mapping
starsCtx.globalCompositeOperation = 'source-over';
for (let i = 0; i < 1200; i++) {
    let size = Math.random() * 0.8 + 0.2;
    starsCtx.globalAlpha = Math.random() * 0.7 + 0.1;

    // Enable hardware-rendered shadows for authentic
glowing effects
    if(size > 0.7) {
        starsCtx.shadowBlur = 4;
        starsCtx.shadowColor = "#ffffff";
    }

    starsCtx.beginPath();
    starsCtx.arc(Math.random() * starsCanvas.width,
Math.random() * starsCanvas.height, size, 0, Math.PI * 2);

```



```
        starsCtx.fill();  
    }  
}
```

Block 4: Web Audio Real-Time Modulator

This module processes physical space boundaries and translates metrics directly into sound waves using a raw audio synthesis oscillator node chain.

```
function initAudio() {  
    if (!audioCtx) {  
        // 1. Establish core hardware context connection  
        audioCtx = new (window.AudioContext ||  
window.webkitAudioContext)();  
  
        // 2. Generate pure synth oscillator node  
        oscillator = audioCtx.createOscillator();  
        oscillator.type = 'sine'; // High-purity tone wave  
  
        // 3. Establish gain architecture interface node for  
sound control  
        gainNode = audioCtx.createGain();  
        gainNode.gain.value = 0; // Mute down safely by  
default  
  
        // 4. Form explicit node connection wire map pipeline  
        oscillator.connect(gainNode);  
        gainNode.connect(audioCtx.destination);  
        oscillator.start();  
    }  
}
```

4. Conclusion & Optimization Protocol

The **Cosmic Choreography** engine balances high-accuracy spatial metrics calculations with optimal interface tracking layouts. By isolating static background graphics from the core drawing elements via layered canvas textures, the application completely eliminates computational redundancy. This design ensures highly responsive rendering on standard consumer mobile browsers and hardware configurations alike.

Project & Maintainer Matrix

- **Developer:** ChandraKamal
- **Email Contact:** chandrakamal303@gmail.com
- **Source Code Repository:**
<https://github.com/Chandrakamal07/Cosmic-Choreography>

© 2026 Kamal. All Rights Reserved.